

Implementing and comparing symmetric-key encryption algorithms

Mark Stone

September 28, 2026

Student Number: 14106463

Programme of Study: BSc Computing

The title of this project has been changed. It was originally "Password Manager - Implementing and comparing symmetric-key encryption algorithms".

This report is the result of my own work except where explicitly stated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged

Abstract

The initial aim of this project was to develop a password manager that let the user choose between several different secure symmetric-key encryption algorithms. The plan was to implement the algorithms AES (also known as Rijndael), Serpent, Twofish, RC6, and MARS.

Due to difficulties with the author's physical and mental health, this did not turn out to be feasible. The final project instead is a simpler program that allows a user to encrypt and decrypt text using two algorithms - AES and Twofish. This project also includes an analysis and comparison of these algorithms' security.

Acknowledgements

I am extremely grateful to my supervisor Dr Dimitrios Airantzis for his invaluable guidance and advice throughout this project.

I am very grateful to Birkbeck's Wellbeing Team for their support regarding my mental health this year.

Finally, I am incredibly grateful to my parents for their love and support throughout this project.

Contents

1	Introduction	1
2	Background	1
3	Original Aims and Objectives	2
3.1	Essential Objectives	2
3.2	Desirable Objectives	3
3.3	Optional Objectives	3
4	Revised Aims and Objectives	4
4.1	Essential Objectives	4
4.2	Desirable Objectives	4
4.3	Optional Objectives	5
5	Methodology and Methods	5
6	Original Design	5
7	Revised Design	6
8	Implementation	8
8.1	AES	8
8.2	Twofish	9
9	Testing and Evaluation	10
10	AES Security	10
11	Twofish Security	11
12	Comparison of AES and Twofish's Performance	12
13	Conclusion	13
A	Github	14
B	Code snippets	14

List of Figures

1	Pages of the application	5
2	Flowchart of revised program	6
3	AES overview	7
4	Twofish overview (Schneier et al. 1998)	8
5	Code to pass a byte through an S-box	14
6	Code to multiply by 2 in $GF(2^8)$	15
7	Code to multiply two numbers in $GF(2^8)$	16

1 Introduction

The original plan for this project was to create a password manager. One aim was to implement several different algorithms to encrypt the database and let the user choose which one they want to use. Another was to analyse these algorithms, comparing their security, their performance (in terms of time taken and memory used), and how straightforward they are to implement.

Due to difficulties with physical and mental health this project had to be simplified. The final plan for this project was to create a program that can encrypt and decrypt multiple secure symmetric-key encryption algorithms. The algorithms AES (Advanced Encryption System, also known as Rijndael) and Twofish were implemented. This report also contains analysis of these algorithms and a comparison of them.

Symmetric-key encryption is useful for a variety of purposes relating to data security. This includes secure communication as well as encrypting data at rest (for example data in a database).

Password managers use symmetric-key encryption to store passwords, because the data is at rest and so there is no advantage to using public-key encryption. Creating a password manager would have been a demonstration of one way symmetric-key encryption is used in practice, but the final result of this project is simply a program that does encryption and decryption.

This report will cover the background to this project. It will then present the original aims and objectives of this project, and the revised aims and objectives once the plan changed. It will explain the methodology used and then present the original and the revised design of the program. After that, details of implementation, testing and evaluation will be presented. It will then finish with the analysis and comparison of AES and Twofish.

2 Background

Symmetric-key encryption algorithms are encryption algorithms where the same key that is used to encrypt the data is also used to decrypt it. This is different to asymmetric-key encryption, where one key is used to encrypt the data and a different key (mathematically related to the first key) is used to decrypt it.

A key is a piece of data, often represented as a string of characters or a binary or hexadecimal number, that is used to encrypt data. Symmetric-key encryption with a well designed algorithm is very secure, as long as the key is kept secret from everyone except the encrypter and authorised decrypter.

Symmetric-key encryption is often used for secure communication after a shared secret key is established using public-key encryption, because symmetric-key encryption is faster than public-key encryption. It is also used to encrypt data that isn't being transmitted, such as data in a database or encrypting the files on a computer.

In 1997, the National Institute of Standards and Technology (NIST) proposed developing a new standard symmetric-key cipher. This was because the

current standard Data Encryption Standard (DES) had become insecure. It had a key size of only 56 bits. In 1997 a DES key was recovered using brute force exhaustion in 96 days, using thousands of computers. In 1998 a key was recovered in a mere 56 hours. Further attacks took even less time. Due to this insecurity, the NIST went through a process starting January 1997 to develop an Advanced Encryption Standard (AES).

From an initial fifteen algorithms, the process narrowed it down to five finalists: MARS, RC6, Rijndael, Serpent, and Twofish. Rijndael was the algorithm selected to be the Advanced Encryption Standard, and so it is now most commonly known as AES (Smid 2021).

A password manager is a program that allows a user to store all their passwords in a secure manner, only accessible by using a master password. The user can then secure their accounts using unique passwords that would be difficult to remember. Password managers use symmetric-key encryption algorithms to encrypt usernames and passwords, so that, if this data is stolen, it is useless without the master password.

Commonly used password managers include Google Password Manager, Apple passwords app, LastPass, Bitwarden and 1Password (Frew & Petrino 2024). Of these password managers, Google Password manager does not disclose how it encrypts data, but the others all use AES-256. None of these allow the user to choose which encryption standard is used. KeePass is an open-source password manager that is similar to the original aim of this project. It allows the user to choose between the encryption algorithms AES, ChaCha20 and Twofish.

3 Original Aims and Objectives

The original aims and objectives of this project were as follows:

The project is creating a password manager that allows the user to choose which secure encryption algorithm they wish to use. The plan was to implement some, potentially all, of the five finalist algorithms from the NIST's Advanced Encryption Standard selection process. The algorithms in question are AES (Advanced Encryption Standard, also known as Rijndael), Serpent, Twofish, RC6, and MARS. In addition, there should be an analysis and comparison of the algorithms implemented in the final report.

3.1 Essential Objectives

- Create a database that stores website URLs, usernames, and passwords
- Create a simple user interface for this application
- A user can set the master password for the application
- A user can log in with the master password
- When a user is logged in, they can save details of a URL, username, and password for a website

- When a user is logged in, they can access a list of URLs and the corresponding usernames and passwords
- When a user is logged in, they can search the database by URL
- When a user is not logged in, they cannot access the information stored in the database
- The passwords in the database are encrypted, the plaintext is not stored.
- The encryption is done using an algorithm that has been implemented manually, without using a pre-existing library or package.
- The user's master password is not stored in plaintext, it is hashed in a secure manner. This will be done using a pre-existing library, not manually implemented.
- A user can log out of the password manager.

3.2 Desirable Objectives

- The user, when creating their account, can choose between multiple encryption algorithms from the following list: AES, Serpent, Twofish, RC6, and MARS. The passwords will be encrypted using that algorithm.
- All the encryption algorithms are implemented manually, without using a pre-existing library or package.
- The final report for this project includes a detailed analysis and comparison of the algorithms that have been implemented. This analysis includes details of how quickly they run when implemented in this project.
- The user can edit entries
- The user can delete entries
- Multiple users can use the application on the same device, each choosing their own username and master password.
- A user can only access the entries associated with their account

3.3 Optional Objectives

- All five algorithms from the Advanced Encryption Standard selection process (AES, Serpent, Twofish, RC6, and MARS) are implemented and can be chosen.
- Alongside URL, username and password, the user can also optionally include a website name
- The search function searches both URL and website name

- The user can also enter notes into a notes section
- The application runs quickly and does not lag
- The user can save multiple accounts for the same website
- The user can export their data to another password manager easily
- The application uses parameterized queries when accessing the database, to prevent SQL injection
- The application will automatically log out after a certain amount of time
- The application will display a short description of each algorithm and their security, when the user is selecting which encryption algorithm to use.
- A user can change their master password.

4 Revised Aims and Objectives

The revised project is creating a program that can encrypt and decrypt multiple secure symmetric-key encryption algorithms. The target algorithms are MARS, RC6, AES, Serpent and Twofish.

4.1 Essential Objectives

- The user can encrypt a message using one of the target encryption algorithms.
- The user can decrypt a message using the same algorithm.
- There is a menu where the user can select if they wish to encrypt or to decrypt a message.

4.2 Desirable Objectives

- More than one encryption algorithm is implemented
- The user can select which algorithm they wish to use
- The user can also exit the program from the menu
- When encrypting, the user can choose between auto-generating a key and typing one manually.
- When a key is auto-generated, it is displayed to the user
- The final report analyses the security and compares the encryption algorithms implemented.

4.3 Optional Objectives

- More than two encryption algorithms are implemented
- There is a simple graphical interface that the user can use

5 Methodology and Methods

This project was completed using the waterfall model. This was a suitable method to use because the design goal was known and it was important to document the work done.

An agile approach would not have been appropriate, as working code without documentation would not have been a suitable end result for this project. There was also not a need to adapt to feedback from a customer, as this project was not developed to meet the needs of a specific customer.

6 Original Design

The original design of the project had this database schema:

Users(*UserID*, *Username*, *MasterPasswordHash*, *Salt*, *EncryptionAlgorithm*)

WebsiteLogins(*URL*, *Username*, *EncryptedPassword*, *Salt*, *UserID*)

It would have four pages:

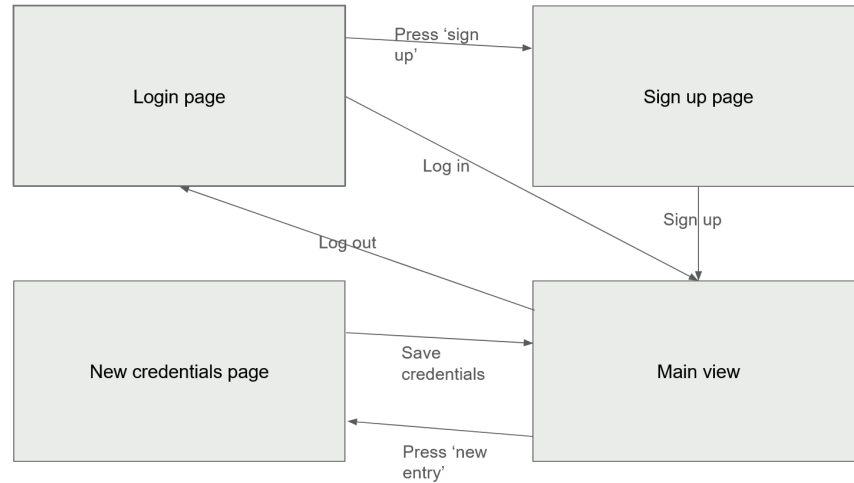


Figure 1: Pages of the application

7 Revised Design

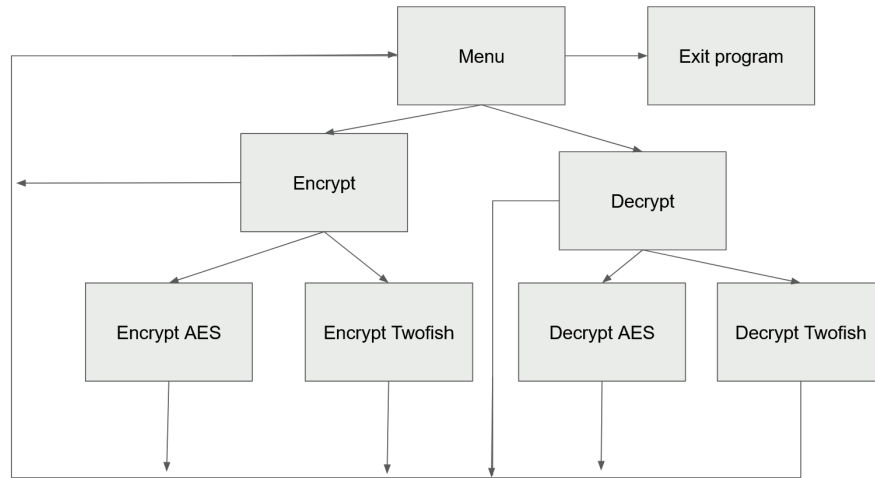


Figure 2: Flowchart of revised program

This is the overall design of the revised program. The menu calls sub-menus which call the encrypt and decrypt functions for AES and Twofish.

The overall design of AES can be seen in Figure 3

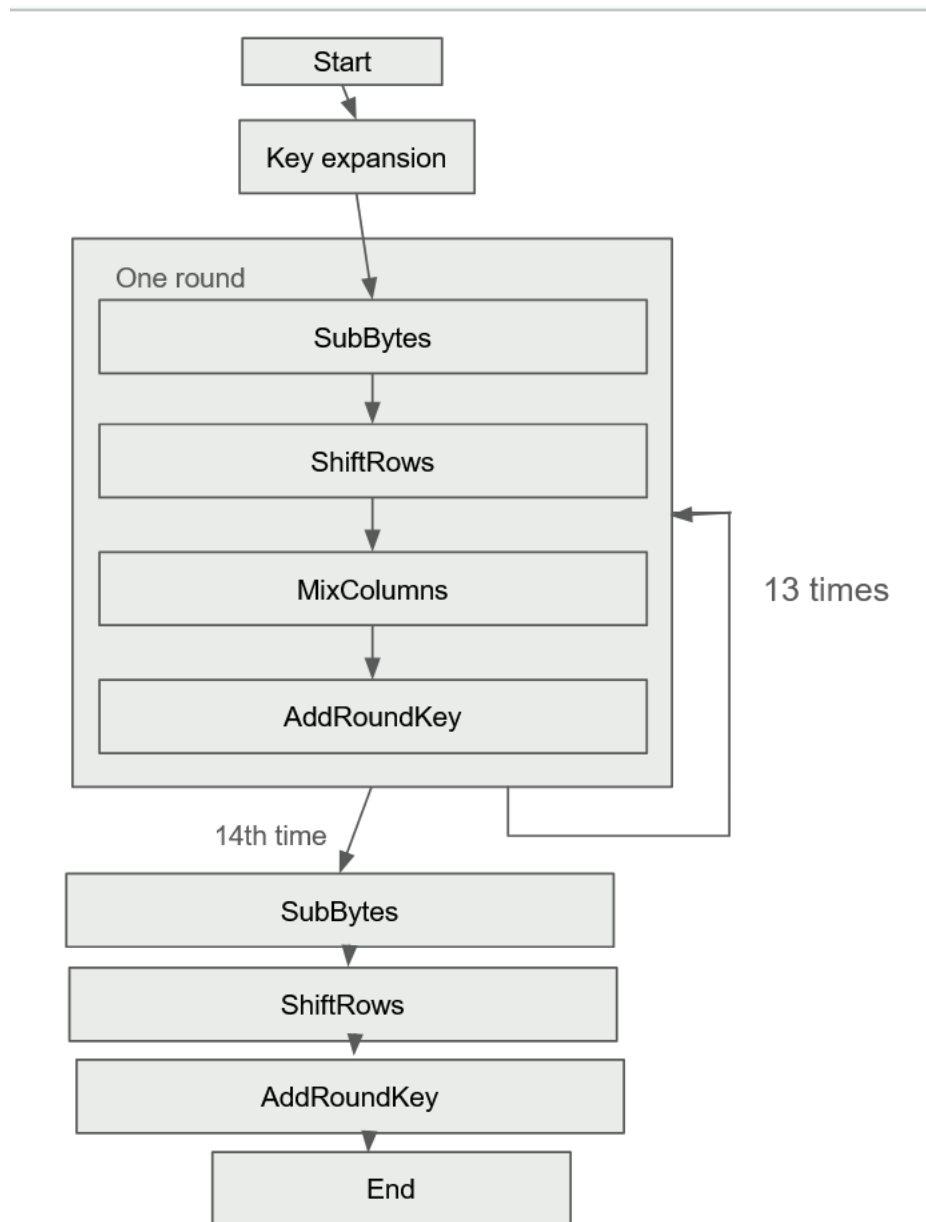


Figure 3: AES overview

The overall design of Twofish can be seen in Figure 4.

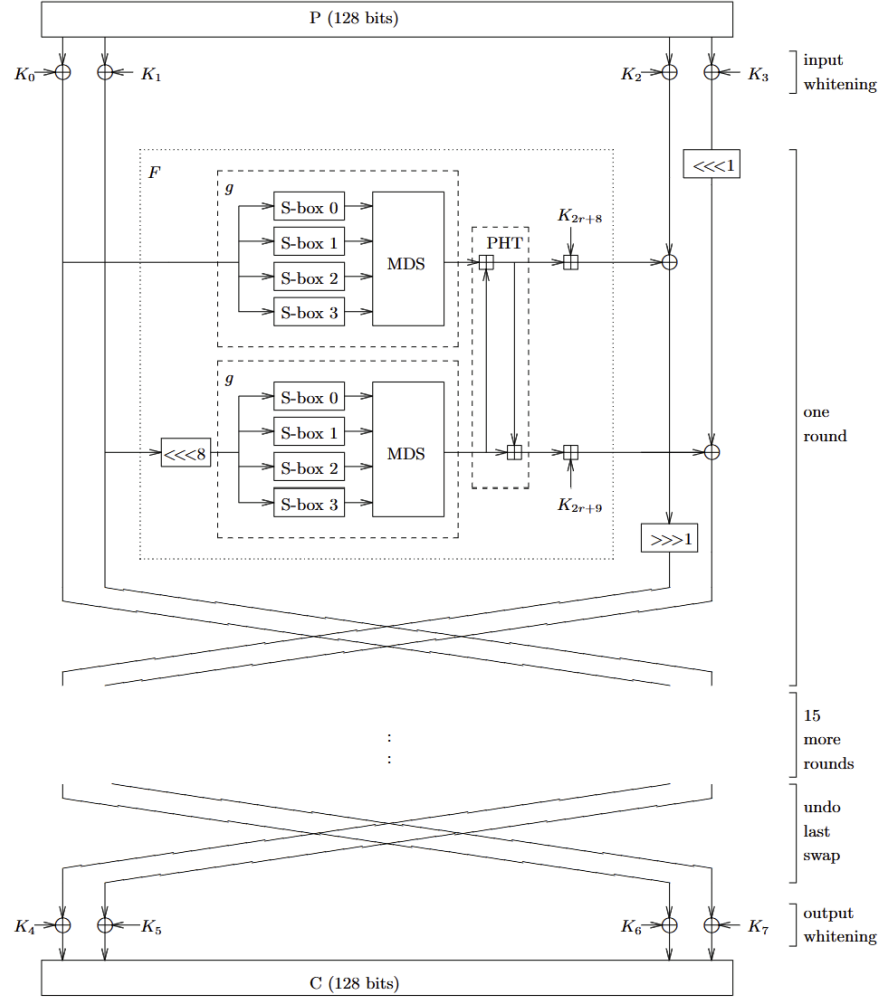


Figure 4: Twofish overview (Schneier et al. 1998)

8 Implementation

8.1 AES

AES has five important functions: `KeyExpansion()`, `SubBytes()`, `ShiftRows()`, `MixColumns()` and `AddRoundKey()`.

`KeyExpansion()` includes using an S-box, rotating a word and XORing it with a constant.

The S-box is a constant 2d array with 16 rows and 16 columns. To transform

a byte using the S-box, the first 4 bits specify the column number and the second four bits specify the row number. The byte in the S-box at this index is returned. See figure 5 in the appendices for the code.

Rotating a word is trivial, as is XORing two numbers.

SubBytes() also puts the bytes through an S-box.

ShiftRows() shifts the first row of the state left by 0, the second by 1, the third by 2 and the 4th by 3. This is simple to implement.

MixColumns() was the most complicated part of AES to implement. Each column of the state is multiplied by a 4x4 matrix constant. However, instead of normal multiplication, it uses multiplication in the Galois Field $GF(2^8)$ with reduction constant $x^8 + x^4 + x^3 + x + 1$. To double a number in this field, first it is multiplied by 2 and then, if it now is greater than 255, it is XORed with the value represented by the reduction constant. $x^8 + x^4 + x^3 + x + 1$ represents the value 100011011 (which in decimal is 283. See figure 6 for the code to perform this doubling.

To multiply in $GF(2^8)$ more generally, the easiest way is to break down one number into powers of 2. For example, $5 = 2^2 + 1$ and $13 = 2^3 + 2^2 + 1$. Therefore, multiplying $x*5$ in $GF(2^8)$, is equivalent to $double(double(x))+x$ and multiplying $x*13$ is equivalent to $double(double(double(x))+double(double(x))+x$. See figure 7 for the code that was used.

$GF(2^8)$ multiplication is also used in Twofish, though with two different reduction constants. Therefore the implementation of Galois Field multiplication is in a separate file (galoisField.rb) from the AES file.

AddRoundKey() just XORs the state with the round key, which is trivial.

To decrypt, SubBytes() uses the inverse S-box, ShiftRows() shifts the rows in the opposite direction and MixColumn() multiplies by a different 4x4 matrix constant.

8.2 Twofish

Twofish calls KeySchedule() and KeyExpansion() before the rounds start. It uses XORs to whiten the input before the first round and after the last round. During the rounds it calls the function f() on half the state, it uses word rotation and XORing with the other half of the state, and then it swaps the two halves. During KeyExpansion() and during f(), the function h() is called.

KeySchedule() divides the key into half, with even bytes going into Meven and odd bytes going into Modd. It then takes the original key in groups of 8 bytes, performing matrix multiplication in $GF(2^8)$ with an 4x8 constant matrix. The group of 8 bytes is treated as an 8x1 vector for this. The result of this is called ArrayS. Meven, Modd and ArrayS are all returned.

KeyExpansion() takes Meven and Modd and generates the expanded key. h() is called twice. The results are mixed using a Pseudo-Hadamard Transform (PHT). This just requires addition and modulo. This is repeated 20 times to generate the expanded key.

f() does some rotation, calls h() and performs a PHT.

In $h()$ a word is split and passed through some fixed S-boxes q_0 and q_1 . It is XORed with a list of words. Finally matrix multiplication in $GF(2^8)$ with a fixed 4x4 matrix occurs.

9 Testing and Evaluation

Tests have been implemented for the program and run correctly. Implementing more tests would be ideal but time pressure caused this to not be feasible. The testing was done using the `rspec` gem.

10 AES Security

AES (originally known as Rijndael) is a block cipher. It encrypts data in blocks of 128 bits. There are three versions of AES: AES-128, AES-192 and AES-256. The numbers refer to the length of the key used to encrypt the plaintext. For AES-128 the number of rounds is 10, for AES-192 it is 12 and for AES-256 it is 14. AES-256 is the strongest of these versions, and in 2015 the NSA (National Security Agency) released guidance in the Commercial National Security Algorithm Suite that AES should be used with 256 bit keys (NSA 2015).

In each round of AES the state passes through three transformations (known as layers). The first layer is the non-linear layer, where each byte of the state is passed through a fixed S-box. This S-box has been designed to have strong non-linearity properties. The next is the linear mixing layer, which is designed to ensure that the algorithm has high diffusion over multiple rounds. The final layer is the key addition layer, where the round key is XORed to the state. This ensures that final output is affected by all of the round keys (Daemen & Rijmen 1999).

In the Report on the Development of the Advanced Encryption Standard (Nechvatal et al. 2001, p. 523) the authors discuss attacks on Rijndael known at that time. These are attacks on versions of Rijndael with a reduced number of rounds, because any successful attacks on the full version developed in the future would likely be based on attacks on a simplified version. The original specification found truncated differential attacks on 4,5 and 6 round versions of Rijndael as well as a collision attack on a 7 round version. Other papers found a way to perform a truncated differential attack on a 7 round version, though with a 192 or 256 bit key the attack did not reduce the search space enough for decryption to be feasible. Attacks on 7 and 8 round Rijndael were found, but they required a very large amount of plaintext and the corresponding ciphertext to already be known to the attacker. Finally an attack on a 9 round version with a 256 bit key is known, but in addition to requiring a large amount of plaintext and their encryptions, it requires a large amount of text with encryptions encrypted using 255 other keys mathematically related to the first key.

Biryukov et al. (2009) found an attack that can use plaintext encrypted with two unknown keys related in a specific way that needs 2^{39} time to recover the

key from a 9 round version of AES-256. It needs 2^{39} plaintexts. This paper also found a chosen ciphertext attack which needs 2^{44} data and 2^{45} time to attack 10 round AES-256. It does not find the whole key, it provides 42 bits of information about the key. Attacks based on related keys are not very practical attacks, as users of AES-256 are unlikely to choose keys that are related in the specific way specified.

Tao & Wu (2015) developed a biclique attack on AES-256 that reduces it from time complexity 2^{256} to $2^{254.27}$, needing 2^{40} data. This is not a very large reduction, only reducing the time needed by a factor of approximately four. It is a key-recovery attack which requires pairs of plaintext and corresponding ciphertext. It doesn't require plaintext encrypted with multiple related keys, or selecting a chosen ciphertext and obtaining the original plaintext, so if it reduced the time sufficiently for finding the key to be feasible it would be a practical attack on AES.

It is believed that AES will remain secure despite improvements in quantum computing. Public-key encryption algorithms such as RSA and Diffie–Hellman key exchange that are secure on classical computers, are vulnerable to Shor's algorithm, which calculates prime factors of an integer. However Shor's algorithm is not useful for deciphering symmetric-key encryption. Decryption of AES could however be sped up by the application of Grover's algorithm. Grover's algorithm is a quantum search algorithm that takes $O(\sqrt{N})$ time to identify a record from a search space that matches a given property (Grover 1996). Grover's algorithm could potentially halve the complexity of AES, reducing AES-256's complexity from 2^{256} to 2^{128} . This is an optimistic estimate as it ignores overheads from parallelisation and error correction (D. & C. 2024). Additionally, a time complexity of 2^{128} is still large enough that it would remain unfeasible to break AES-256. Therefore, AES-256 is considered to be quantum resistant.

11 Twofish Security

Twofish is also a block cipher that encrypts plaintext in blocks of 128 bits. It also has three versions: one with a 128 bit key, one with a 192 bit key and one with a 256 bit key. Twofish always runs for 16 rounds, no matter the key size.

It is designed with a Feistel-like structure. This means that the rounds consist of the round function being run on one half of the state, the second half being XORed with the first, and then the two halves being switched. A Feistel structure was chosen because Feistel networks were well studied, and using a more novel structure would increase the chance of there being flaws with the structure that hadn't been identified.

Whitening is done before the first round and after the last round. This means XORing the state with part of the key, and makes key-search attacks significantly harder.

Twofish uses key-dependent S-boxes. This means that the S-boxes are different when different keys are used. In a Twofish round the first half of the state

is passed through the key-dependent S-boxes, a Pseudo-Hadamard Transform (PHT) is performed and then it is multiplied with a Maximum Distance Separable (MDS) matrix. The PHT and MDS matrix are both intended to increase diffusion (Schneier et al. 1998).

In the Report on the Development of the Advanced Encryption Standard (Nechvatal et al. 2001, p. 523) the authors discuss attacks on reduced round versions of Twofish. An impossible-differential attack can be used to break 6 rounds of Twofish encrypted with a 256 bit key, it takes 2^{256} steps to do so. If the whitening step is removed, this can be extended to 7 round Twofish.

Ortiz & Compton (2016) found that a side-channel attack using Simple Power Analysis (SPA) can break Twofish running in an 8-bit environment. Furthermore this is likely to extend to 32-bit and 64-bit implementations too. A SPA attack relies on information gained by analysing how electricity use of the machine running Twofish fluctuates as the algorithm runs. This means that this attack is effective when an attacker has physical access to the device on which Twofish is running on.

There are not many attacks on Twofish published after the NIST’s decision to select Rijndael to become the Advanced Encryption System. This is unlikely to reflect Twofish being significantly more secure than AES. Instead, it is likely that fewer people worked on attempting to find flaws in Twofish because AES became the main symmetric-key encryption algorithm used.

There appears to be an absence of academic writing that analyses Twofish’s resistance to attacks using quantum computers. Nevertheless, as it does not rely on factoring large integers being difficult, it should not be vulnerable to Shor’s algorithm. As Grover’s algorithm is a general search algorithm, Twofish would be vulnerable to it in the same way AES is, with it potentially halving the complexity of the encryption. As there has not been significant work done to identify if other forms of attack with quantum computers would cause Twofish to be broken, it is unclear whether Twofish is truly quantum resistant or not.

12 Comparison of AES and Twofish’s Performance

Dibas & Sabri (2021) compared the performance of AES and Twofish (as well as algorithms 3DES and Blowfish) and found that AES had faster encryption times and used less memory. Rizvi et al. (2011) found that increasing RAM size increased the performance of Twofish more than it did AES, and that with sufficient RAM Twofish performed better than AES when encrypting text or sound. G & S (2024) found that AES had significantly lower encryption and decryption times than Twofish, and thus had a much higher data throughput.

While the AES selection process was ongoing, Schneier & Whiting (2000) found that the performance of AES and Twofish was fairly similar. Under some tests AES performed better but under others Twofish did. They concluded that there was no obvious answer to which algorithm would perform better overall

in real life conditions.

Subsequently to this, hardware acceleration has been developed for AES. For example, Intel created an AES instruction set in 2010 to speed up encryption and decryption using AES (Gueron 2010). It is likely that AES performing better than Twofish in recent comparisons is due to improvements in hardware acceleration of AES.

Overall, AES performs better than Twofish in recent comparisons. It is likely that if Twofish had been selected to become the Advanced Encryption Standard, then hardware acceleration for Twofish would be more developed and used, meaning Twofish would have better performance. Nevertheless, as things currently stand, AES has the advantage over Twofish.

13 Conclusion

This project has created a program that can encrypt and decrypt AES and Twofish. This is a reduced version of the original plan to create a password manager. It has also analysed and compared AES and Twofish. In recent years AES performs significantly better than Twofish. In addition, the security of AES has been more thoroughly scrutinised and attacked, as it is much more commonly used. It is unknown if the same level of scrutiny on Twofish would reveal practical attacks that could break Twofish. As we can be more confident about the security of AES and it performs better, AES is the better symmetric-key algorithm to use.

One avenue of future research that hasn't been done before would be to attempt to find methods of breaking Twofish using algorithms run on quantum computers. Another would be to follow up on this project's original goal, particularly creating a password manager that can use MARS, RC6 and Serpent.

A Github

The code for my project is accessible at <https://github.com/M-Stone941/TwoFactorAuthentication>

B Code snippets

```
def sbox(byte, inverse)
  column = byte.div(16)
  row = byte.modulo(16)
  if inverse==False
    result = SBOXDECIMAL[column][row]
  else
    result = INVERSESBOXDECIMAL[column][row]
  end
  result
end
```

Figure 5: Code to pass a byte through an S-box

```

M-Stone941
def galoisDoubler(number, constant="AES")
  if constant == "AES"
    reductionConstant = AESGaloisFieldReductionConstant
  elsif constant == "TwofishG"
    reductionConstant = TwofishGReductionConstant
  elsif constant == "TwofishKey"
    reductionConstant = TwofishKeyScheduleReductionConstant
  else
    puts "Error - no galois field reduction constant specified"
  end
  # multiplies by two in the Galois field 2^8
  if number.div(128)==0
    number*2
  else
    (number*2)^reductionConstant
  end
end
end

```

Figure 6: Code to multiply by 2 in $GF(2^8)$

```

M-Stone941
def galoisTimes(number1, number2, constant="AES")
  # multiply two numbers in the galois field 2^8
  sum = 0
  index = 0
  while number2 > 0
    remainder = number2 % 2
    if remainder == 1
      temp = number1
      index.times do | Integer i |
        temp = galoisDoubler(temp, constant)
      end
      sum = sum^temp
    end
    number2 = number2 / 2
    index += 1
  end
  sum
end

```

Figure 7: Code to multiply two numbers in $GF(2^8)$

References

- Biryukov, A., Dunkelman, O., Keller, N., Khovratovich, D. & Shamir, A. (2009), 'Key recovery attacks of practical complexity on AES variants with up to 10 rounds', Cryptology ePrint Archive, Paper 2009/374.
URL: <https://eprint.iacr.org/2009/374>
- D., S. & C., P. (2024), 'On the practical cost of grover for aes key recovery'.
URL: <https://csrc.nist.gov/csrc/media/Events/2024/fifth-pqc-standardization-conference/documents/papers/on-practical-cost-of-grover.pdf>
- Daemen, J. & Rijmen, V. (1999), 'Aes proposal: Rijndael'.
URL: <https://csrc.nist.gov/encryption/aes/rijndael/Rijndael.pdf>

- Dibas, H. & Sabri, K. E. (2021), A comprehensive performance empirical study of the symmetric algorithms:aes, 3des, blowfish and twofish, *in* ‘2021 International Conference on Information Technology (ICIT)’, pp. 344–349.
- Frew, P. & Petrino, G. (2024), ‘2024 password manager industry report and statistics’.
URL: <https://www.security.org/digital-safety/password-manager-annual-report/>
- G, M. M. & S, S. (2024), Securing cloud data: A comprehensive review of hybrid cryptography and analysis of aes, blowfish, and twofish algorithms, *in* ‘2024 3rd International Conference on Automation, Computing and Renewable Systems (ICACRS)’, pp. 568–575.
- Grover, L. K. (1996), A fast quantum mechanical algorithm for database search, *in* ‘Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing’, STOC ’96, Association for Computing Machinery, New York, NY, USA, p. 212–219.
URL: <https://doi.org/10.1145/237814.237866>
- Gueron, S. (2010), Intel® advanced encryption standard (aes) new instructions set, Technical report, Intel Corporation.
- Nechvatal, J., Barker, E., Bassham, L., Burr, W., Dworkin, M., Foti, J. & Roback, E. (2001), ‘Report on the development of the advanced encryption standard (aes)’, *Journal of Research of the National Institute of Standards and Technology*.
- NSA (2015), ‘Commercial national security algorithm suite’.
URL: <https://web.archive.org/web/20220218193742/https://apps.nsa.gov/iaarchive/programs/iad-initiatives/cnsa-suite.cfm>
- Ortiz, J. J. G. & Compton, K. J. (2016), ‘A simple power analysis attack on the twofish key schedule’.
URL: https://josejg.com/papers/twofish_tfg.pdf
- Rizvi, S., Hussain, S. Z. & Wadhwa, N. (2011), Performance analysis of aes and twofish encryption schemes, *in* ‘2011 International Conference on Communication Systems and Network Technologies’, pp. 76–79.
- Schneier, B., Kelsey, J., Whiting, D., Wagner, D., Hall, C. & Ferguson, N. (1998), ‘Twofish: A 128-bit block cipher’.
URL: <https://www.schneier.com/wp-content/uploads/2016/02/paper-twofish-paper.pdf>
- Schneier, B. & Whiting, D. (2000), A performance comparison of the five aes finalists, *in* ‘Proceedings of the Third AES Candidate Conference’.

- Smid, M. E. (2021), ‘Development of the advanced encryption standard’, *Journal of Research of the National Institute of Standards and Technology* .
- Tao, B. & Wu, H. (2015), Improving the biclique cryptanalysis of aes, *in* E. Foo & D. Stebila, eds, ‘Information Security and Privacy’, Springer International Publishing, Cham, pp. 39–56.